

Trabalho Prático 01 – Algoritmos II

Gustavo Luiz A. R.
2020035116

Pedro H. M. G. Cortez
2024013451

1 Introdução

Este trabalho tem como objetivo desenvolver uma aplicação interativa para consulta espacial de bares participantes do Comida di Buteco de 2026 em Belo Horizonte. A proposta consiste em permitir que o usuário informe um endereço e um valor de alcance, utilizado para definir uma região de interesse no mapa. A partir dessa entrada, o sistema deve identificar quais bares estão localizados dentro da área definida e exibir os resultados de forma visual e organizada.

Para isso, os endereços dos bares são convertidos em coordenadas geográficas, representadas por latitude e longitude. Esses pontos são então organizados em uma árvore k -dimensional, ou *k-d tree*, estrutura utilizada para realizar buscas ortogonais eficientes em conjuntos de pontos. No caso principal do trabalho, a busca é feita em uma região retangular, cujo tamanho é determinado pela diagonal informada pelo usuário.

A aplicação também conta com uma interface web desenvolvida em Python, utilizando Dash e Dash Leaflet, permitindo a visualização dos bares em um mapa interativo. Os bares encontrados são exibidos como marcadores no mapa e também em uma tabela contendo nome, endereço e distância em relação ao endereço pesquisado. Como funcionalidade adicional, foi implementada uma busca circular, na qual o usuário pode consultar bares dentro de um raio determinado.

Dessa forma, o projeto integra conceitos de geometria computacional, estruturas de dados espaciais, geocodificação e visualização interativa de dados geográficos, aproximando o conteúdo teórico da disciplina de uma aplicação prática e realista.

A aplicação foi disponibilizada publicamente para acesso pela web, permitindo a utilização do sistema sem necessidade de instalação local. O sistema pode ser acessado em: <https://tp1-alg2-6f51.onrender.com/>.

2 Modelagem

A modelagem do sistema foi dividida em duas partes principais: *backend* e *frontend*. Essa separação foi adotada para manter a lógica algorítmica independente da camada de visualização.

O *backend* é responsável por carregar os dados dos bares, armazenar suas coordenadas, construir a árvore k -dimensional, realizar as buscas espaciais e calcular as distâncias entre o endereço informado e os bares encontrados. Já o *frontend* é responsável pela interação com o usuário, exibição do mapa, desenho da região de busca, apresentação dos marcadores e construção da tabela de resultados.

Cada bar é representado por um ponto geográfico:

$$p = (\text{latitude}, \text{longitude})$$

Esses pontos são utilizados como entrada para a construção da k-d tree. Além das coordenadas, cada bar também possui informações textuais, como nome, rua, número, bairro, cidade e CEP. Essas informações são usadas na tabela e nos popups exibidos no mapa.

O endereço informado pelo usuário também é convertido em coordenadas geográficas. Esse ponto passa a ser o centro da região de busca. Na busca retangular, o usuário informa a diagonal do retângulo. Na busca circular, o mesmo campo é utilizado como raio.

3 Implementação

A solução desenvolvida segue um fluxo principal. Inicialmente, os endereços dos bares são convertidos em coordenadas geográficas. Em seguida, esses pontos são organizados em uma k-d tree. Quando o usuário realiza uma busca, o endereço digitado é geocodificado, a região de interesse é calculada e a árvore é consultada para retornar apenas os bares dentro da área.

A base de dados dos bares contém endereços textuais, mas não necessariamente coordenadas geográficas prontas. Por isso, foi utilizada a biblioteca GeoPy com o serviço Nominatim, baseado no OpenStreetMap. A geocodificação transforma um endereço textual em um par de coordenadas de latitude e longitude.

Para evitar excesso de chamadas ao serviço externo, foi utilizado um mecanismo de controle de taxa com `RateLimiter`. Além disso, o sistema pode reutilizar coordenadas já obtidas anteriormente, reduzindo a necessidade de novas requisições.

3.1 K-d tree

A k-d tree foi implementada manualmente. Cada nó da árvore armazena um ponto e referências para os filhos esquerdo e direito. Como o problema possui duas dimensões, os eixos são alternados entre latitude e longitude a cada nível da árvore.

Nos níveis pares, a divisão é feita pela latitude. Nos níveis ímpares, a divisão é feita pela longitude. Em cada chamada recursiva, os pontos são ordenados pelo eixo atual, o ponto mediano é escolhido como raiz da subárvore e os demais pontos são divididos entre as subárvores esquerda e direita.

Essa estratégia tende a manter a árvore relativamente balanceada, permitindo consultas espaciais mais eficientes do que uma busca sequencial sobre todos os bares.

3.2 Busca Retangular

A busca retangular é a principal funcionalidade do trabalho. O usuário informa um endereço e o comprimento da diagonal da região de interesse. Esse endereço é convertido em coordenadas geográficas e passa a representar o centro da região de busca. A partir desse ponto central, o sistema constrói uma área retangular ao redor do endereço pesquisado.

No projeto, a região foi tratada como um retângulo regular, isto é, um quadrado centralizado no ponto informado pelo usuário. Assim, a diagonal fornecida determina diretamente o tamanho da área de busca. A partir da diagonal d , calcula-se o lado do retângulo:

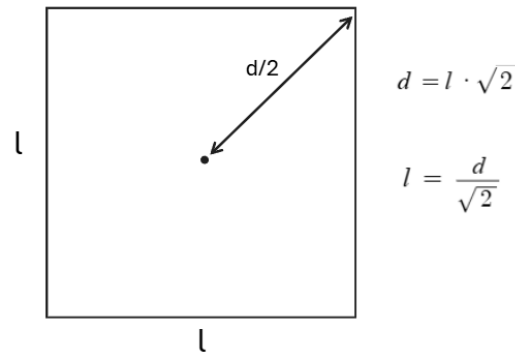


Figura 1: Cálculo do lado do retângulo

Como o endereço pesquisado representa o centro da região, o sistema utiliza metade do lado calculado para determinar até onde o retângulo deve se estender em cada direção. Dessa forma, são obtidos quatro limites: latitude mínima, latitude máxima, longitude mínima e longitude máxima. Esses valores definem a região ortogonal da busca.

Depois de calculada a região, a busca é realizada na k-d tree. A cada nó visitado, o sistema verifica se o ponto armazenado naquele nó está dentro dos limites do retângulo. Para isso, a latitude do ponto precisa estar entre a latitude mínima e a latitude máxima, e a longitude precisa estar entre a longitude mínima e a longitude máxima. Quando essas duas condições são satisfeitas, o ponto é considerado pertencente à região de busca.

A vantagem da k-d tree aparece no momento de decidir quais subárvores precisam ser exploradas. Como cada nível da árvore divide o espaço por um eixo específico, latitude ou longitude, é possível verificar se a região retangular cruza ou não o plano de divisão daquele nó. Se a região de busca estiver totalmente de um lado da divisão, a subárvore do lado oposto pode ser descartada, pois nenhum ponto nela poderá pertencer ao retângulo. Caso a região atravesse a divisão, as duas subárvores podem precisar ser analisadas.

Esse processo reduz o número de pontos examinados quando comparado a uma busca sequencial em todos os bares. Em vez de verificar todos os estabelecimentos cadastrados, o algoritmo percorre apenas os ramos da árvore que possuem possibilidade de conter pontos dentro da região definida pelo usuário. Assim, a k-d tree atua como uma estrutura de organização espacial que permite podar partes irrelevantes do conjunto de dados.

Ao final da consulta, a árvore retorna as coordenadas dos bares que estão dentro do retângulo. Essas coordenadas são então associadas novamente às informações completas dos estabelecimentos, como nome, endereço, bairro e CEP. Em seguida, o sistema calcula a distância entre cada bar encontrado e o endereço informado pelo usuário, ordenando os resultados em ordem crescente de distância.

3.3 Busca Circular

Como funcionalidade adicional, foi implementada uma busca circular. Essa funcionalidade permite ao usuário consultar bares dentro de um raio definido a partir do endereço informado. Para sua implementação, foi utilizada como referência a abordagem apresentada no livro *Advanced Algorithms and Data Structures*, de Marcello La Rocca, especialmente a seção 9.3.6, que trata de *Region Search* em regiões hiperesféricas.

Nesse modo de busca, o valor informado pelo usuário é interpretado como o raio da região circular, em quilômetros. O endereço digitado é convertido em coordenadas geográficas e passa a representar o centro do círculo. A partir desse centro e do raio informado, o objetivo passa a ser encontrar todos os bares cuja distância até o ponto central seja menor ou igual ao raio definido.

Como os pontos armazenados na k-d tree estão representados por latitude e longitude, não é possível utilizar diretamente o raio em quilômetros como medida dentro da árvore. Por isso, inicialmente é feita uma aproximação do raio em graus. Essa aproximação considera a conversão média entre quilômetros e graus de latitude e também leva em conta a variação da longitude conforme a latitude do ponto central. Com isso, obtém-se uma região aproximada que permite consultar a k-d tree e recuperar pontos candidatos.

A busca circular na k-d tree funciona de forma semelhante à busca por região, mas utilizando a distância aproximada em relação ao centro como critério. Em cada nó visitado, o algoritmo calcula se o ponto armazenado está dentro da região aproximada. Além disso, a árvore permite descartar subárvores que estão suficientemente distantes do centro em relação ao eixo de divisão atual. Assim, quando a diferença entre o centro da busca e o ponto do nó indica que uma subárvore não pode conter pontos dentro do raio, essa subárvore não precisa ser percorrida.

Entretanto, essa primeira etapa ainda trabalha com uma aproximação baseada em graus de latitude e longitude. Como essas coordenadas não representam distâncias lineares uniformes na superfície terrestre, especialmente no caso da longitude, alguns pontos retornados pela k-d tree podem estar ligeiramente fora do raio real informado pelo usuário. Para resolver isso, foi aplicada uma segunda etapa de filtragem.

Nessa etapa final, cada ponto candidato retornado pela k-d tree tem sua distância real até o endereço pesquisado calculada pela fórmula de Haversine. Essa fórmula considera a curvatura da Terra e retorna a distância aproximada entre dois pontos geográficos em quilômetros. Apenas os bares cuja distância real é menor ou igual ao raio informado permanecem no resultado final.

Após a filtragem, os bares encontrados são ordenados em ordem crescente de distância em relação ao endereço pesquisado.

3.4 Interface Web

A interface web foi construída com Dash e Dash Leaflet, permitindo que o usuário interaja diretamente com o mapa e com os filtros de busca. Ao abrir o sistema, todos os bares cadastrados são exibidos no mapa por meio de marcadores, enquanto a tabela de resultados permanece vazia. Essa decisão foi tomada para que o usuário tenha uma visão geral da distribuição dos bares antes de realizar uma consulta específica.

Quando o usuário informa um endereço e um valor de alcance, o sistema executa a busca espacial no backend e atualiza dinamicamente os elementos da interface. Nesse

momento, o mapa passa a exibir apenas os bares localizados dentro da área definida, e a tabela abaixo do mapa é preenchida com os resultados encontrados em ordem crescente de distância. O sistema também desenha a região de busca no mapa, podendo ser um retângulo ou um círculo, de acordo com a opção selecionada pelo usuário.

Além da funcionalidade básica de exibição do mapa, foram feitas customizações visuais para melhorar a experiência de uso. O mapa padrão utiliza o estilo Voyager da CartoDB, baseado em dados do OpenStreetMap e com visual limpo, adequado para destacar os marcadores e a área de busca. Também foi implementada uma camada alternativa de satélite, utilizando o Alidade Satellite da Stadia Maps. Dessa forma, o usuário pode alternar entre uma visualização cartográfica tradicional e uma visualização por imagem de satélite.

Também foram utilizados marcadores personalizados para representar os bares e o ponto pesquisado pelo usuário. Os bares são exibidos com ícones temáticos relacionados ao Comida di Buteco, enquanto o endereço pesquisado recebe um marcador próprio, visualmente distinto dos demais. Essa diferenciação facilita a identificação do centro da busca e evita confusão entre o ponto informado pelo usuário e os bares retornados pelo sistema.

Outro recurso implementado foi a camada de divisão dos bairros de Belo Horizonte. Para isso, foi utilizado um arquivo GeoJSON com os limites dos bairros, exibido sobre o mapa por meio do componente `d1.GeoJSON`. Essa camada pode ser ativada ou desativada pelo usuário, permitindo visualizar melhor a organização espacial da cidade sem sobrecarregar permanentemente o mapa.

A interface também possui botões adicionais para controle da visualização, como o botão para alternar a exibição dos bairros e o botão para trocar entre o mapa padrão e o mapa de satélite. Além disso, os popups dos bares foram personalizados para apresentar o nome, endereço e CEP de forma mais clara e visualmente integrada ao restante da aplicação.

Por fim, a tabela de resultados foi estilizada para melhorar a legibilidade, com cabeçalho destacado, linhas alternadas em tons claros e coluna de distância em evidência. Assim, a interface combina visualização espacial e listagem textual, permitindo que o usuário compreenda rapidamente quais bares estão dentro da região definida e quais são os mais próximos do endereço pesquisado.

4 Exemplos de Funcionamento

4.1 Tela Inicial



Figura 2: Tela inicial da aplicação com todos os bares exibidos no mapa.

Ao abrir a aplicação, todos os bares cadastrados são exibidos no mapa. A tabela permanece vazia até que uma busca seja realizada.

4.2 Busca Retangular

Na busca retangular, o usuário informa um endereço e o comprimento da diagonal da região de busca. O sistema desenha o retângulo no mapa e exibe apenas os bares localizados dentro dele.



Figura 3: Exemplo de busca retangular no mapa e tabela com os bares encontrados.

4.3 Busca Circular

Na busca circular, o valor informado é tratado como raio. A aplicação desenha um círculo ao redor do endereço pesquisado e retorna os bares localizados dentro dessa distância.

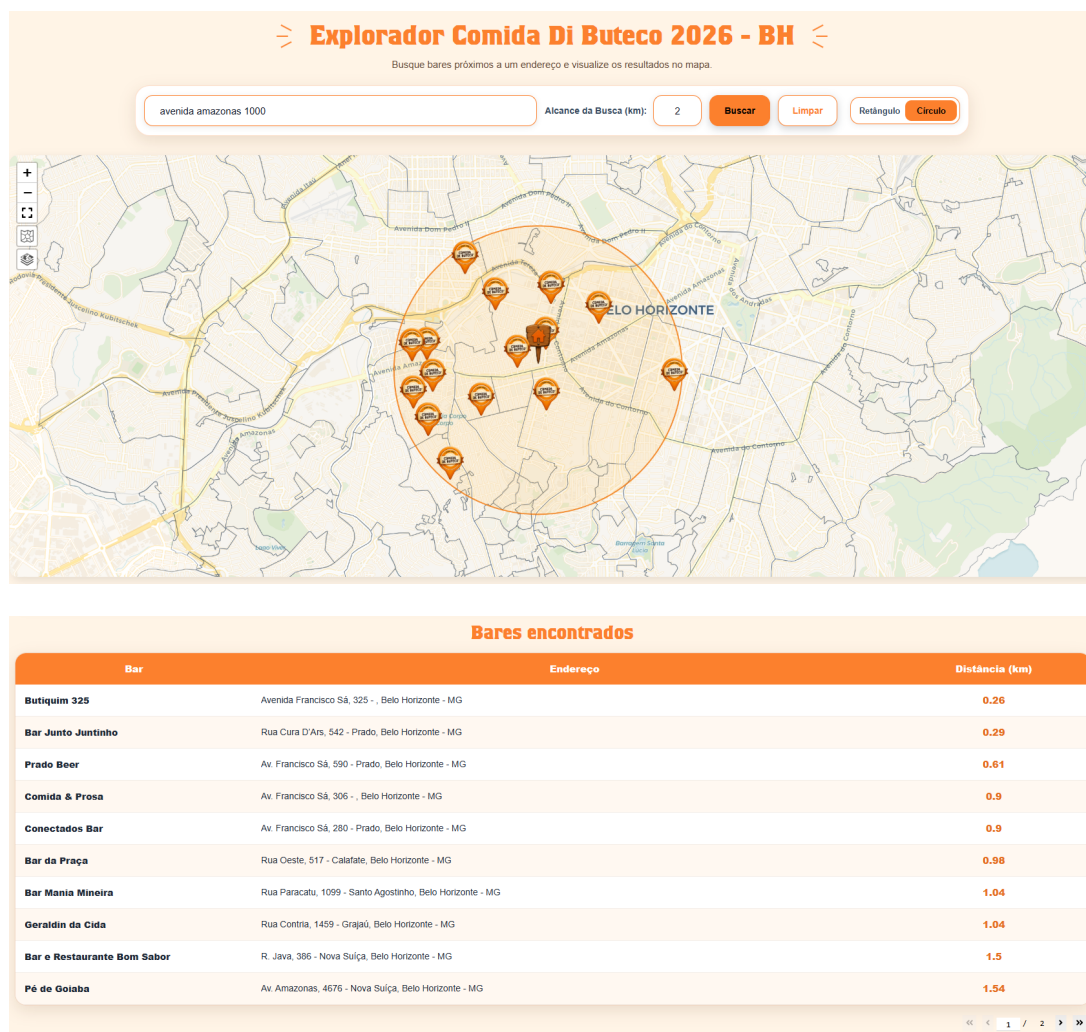


Figura 4: Exemplo de busca circular no mapa e tabela com os bares encontrados.

5 Implementações Extras

Além das funcionalidades principais exigidas no trabalho, foram implementados alguns recursos adicionais com o objetivo de melhorar a experiência de uso e a apresentação visual do sistema. Esses recursos incluem a alternância entre mapa padrão e mapa de satélite, a exibição opcional dos contornos dos bairros de Belo Horizonte e a personalização dos marcadores e popups dos bares.

Foi implementado um botão no mapa que permite alternar entre duas camadas de visualização. A primeira utiliza o estilo Voyager da CartoDB, que apresenta um mapa claro e adequado para destacar os marcadores dos bares. A segunda utiliza a camada Alidade Satellite da Stadia Maps, oferecendo uma visualização por imagem de satélite.

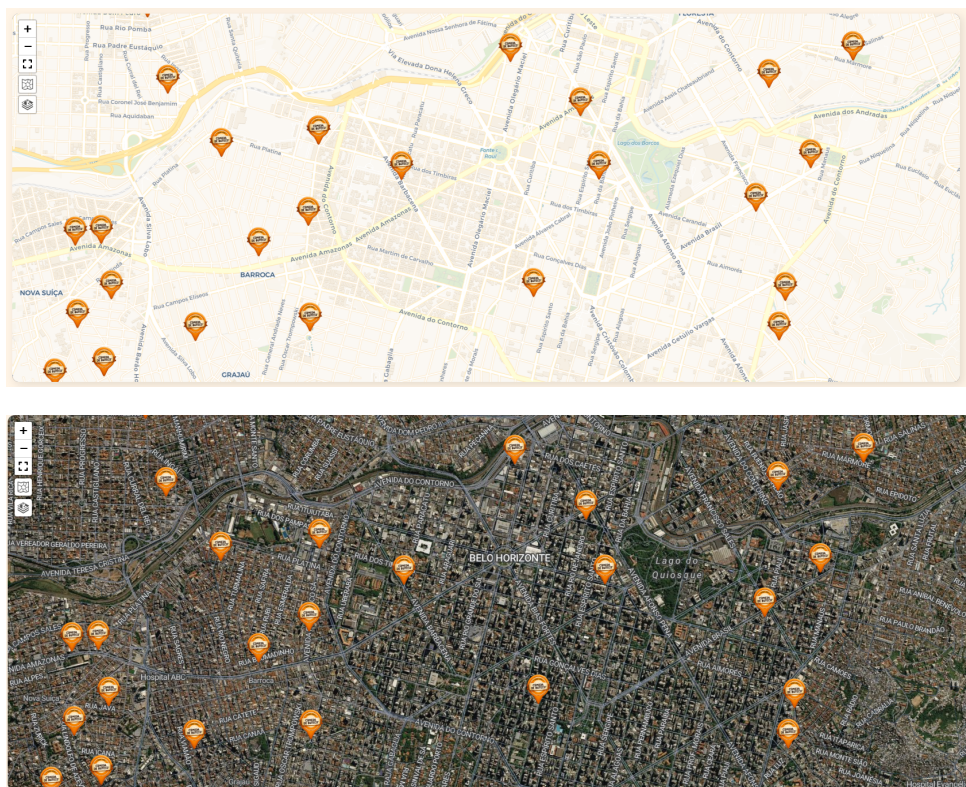


Figura 5: Comparação entre o mapa padrão e a visualização por satélite.

Também foi adicionada uma camada com os limites dos bairros de Belo Horizonte. Essa camada é carregada a partir de um arquivo GeoJSON e pode ser ativada ou desativada pelo usuário por meio de um botão no mapa. Esse recurso ajuda a interpretar melhor a distribuição espacial dos bares em relação às regiões da cidade.

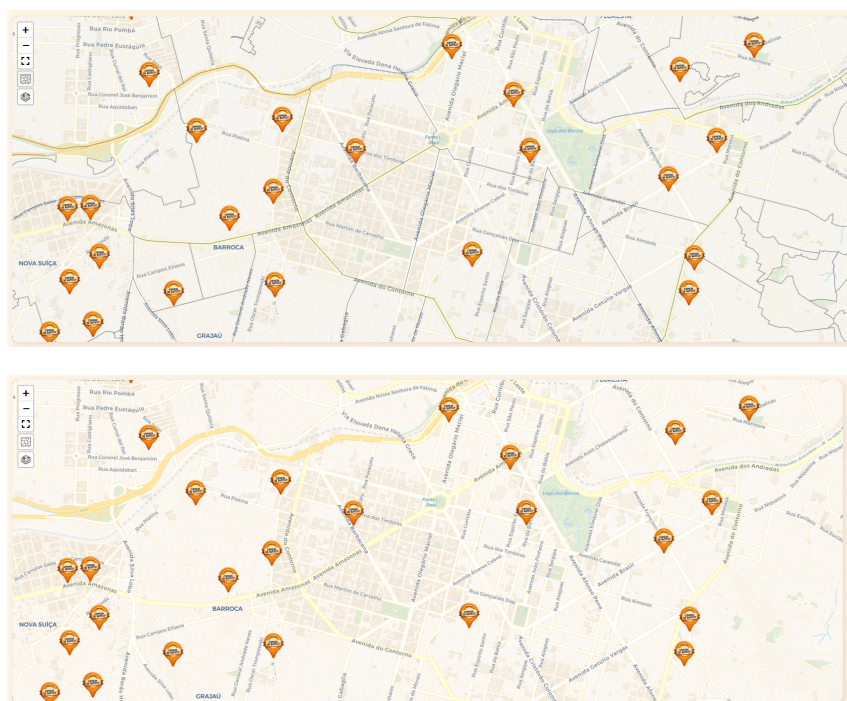


Figura 6: Comparação entre o mapa sem contornos e com os limites dos bairros ativados.

Os marcadores dos bares foram personalizados com ícones temáticos relacionados ao Comida di Buteco. Além disso, o popup exibido ao clicar em um marcador também foi estilizado, mostrando o nome do bar em destaque, o endereço e o CEP. Essa alteração melhora a leitura das informações e deixa a interface mais integrada à identidade visual do projeto.

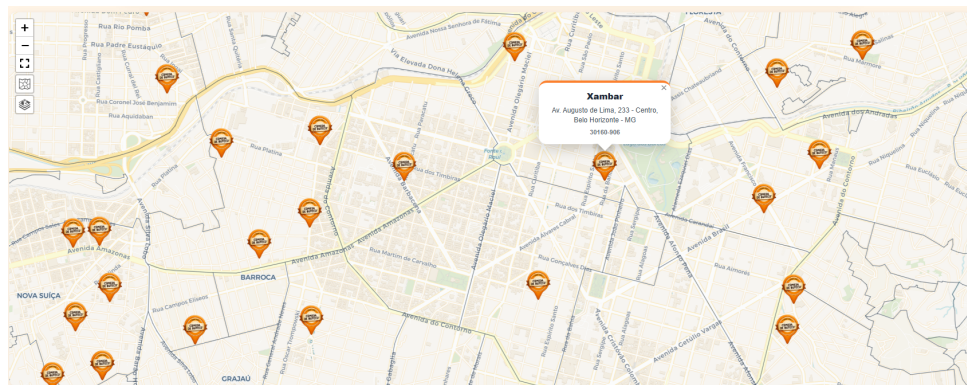


Figura 7: Exemplo de marcador personalizado com popup exibindo as informações do bar.

6 Considerações Finais

A realização deste trabalho permitiu aplicar conceitos de geometria computacional e estruturas de dados espaciais em uma aplicação prática e interativa. A implementação da k-d tree possibilitou organizar os bares participantes do Comida di Buteco a partir de suas coordenadas geográficas, utilizando latitude e longitude como dimensões do espaço. Com isso, foi possível realizar buscas por região de forma mais estruturada do que uma varredura simples sobre todos os pontos.

O sistema desenvolvido atende ao objetivo principal do trabalho ao permitir que o usuário informe um endereço, defina o tamanho de uma região retangular por meio da diagonal e visualize os bares contidos nessa área. Além disso, os resultados são exibidos tanto no mapa quanto em uma tabela ordenada pela distância em relação ao endereço pesquisado, tornando a consulta mais clara e útil para o usuário.

Além da busca retangular, também foi implementada uma funcionalidade extra de busca circular. Nesse caso, o valor informado pelo usuário é interpretado como raio, permitindo consultar bares dentro de uma região circular. Essa funcionalidade exigiu uma etapa adicional de filtragem pela distância real em quilômetros, calculada pela fórmula de Haversine, garantindo maior precisão nos resultados retornados.

Outro ponto importante foi a integração entre diferentes partes do sistema. O projeto envolveu geocodificação de endereços, manipulação de dados geográficos, construção de uma estrutura de dados espacial, desenvolvimento de interface web e customização visual do mapa. Essa integração tornou o trabalho mais próximo de uma aplicação real, indo além da implementação isolada de um algoritmo.

A separação entre backend e frontend também foi uma decisão relevante. O backend concentrou a lógica de geocodificação, cálculo de distâncias, construção da k-d tree e execução das buscas. O frontend ficou responsável pela interação com o usuário, exibição do mapa, desenho das regiões de busca, marcadores personalizados e tabela de resultados.

Essa organização facilitou a manutenção do código e deixou mais clara a responsabilidade de cada parte do projeto.

Por fim, o trabalho contribuiu para fixar os conceitos estudados na disciplina de Algoritmos II, especialmente no que se refere ao uso de estruturas de dados para problemas geométricos. A aplicação final demonstra como uma k-d tree pode ser utilizada em um contexto prático de consulta espacial, permitindo ao usuário explorar dados geográficos de forma visual, interativa e intuitiva.

Referências

- [1] COMIDA DI BUTECO. *Butecos participantes – Belo Horizonte*. Disponível em: <https://comidadibuteco.com.br/butecos/belo-horizonte/>. Acesso em: 2 maio 2026.
- [2] PLOTLY. *Dash Documentation*. Disponível em: <https://dash.plotly.com/>. Acesso em: 2 maio 2026.
- [3] DASH LEAFLET. *Dash Leaflet Documentation*. Disponível em: <https://www.dash-leaflet.com/>. Acesso em: 2 maio 2026.
- [4] GEOPY. *GeoPy Documentation*. Disponível em: <https://geopy.readthedocs.io/>. Acesso em: 2 maio 2026.
- [5] OPENSTREETMAP. *OpenStreetMap*. Disponível em: <https://www.openstreetmap.org/>. Acesso em: 2 maio 2026.
- [6] NOMINATIM. *Nominatim Documentation*. Disponível em: <https://nominatim.org/>. Acesso em: 2 maio 2026.
- [7] LA ROCCA, Marcello. *Advanced Algorithms and Data Structures*. Manning Publications, 2021.